

Four Hands-On Builds for Vision, MLOps and Feedback Loops

Four independent builds on an Azure virtual machine with GitHub Copilot — diagnosing a vision model before improving it, then building the operational machinery that keeps improvement running.

Companion sessions	Module 1 — Computer vision for industrial use cases · Module 2 — CV error analysis and hypothesis-driven improvement · Module 3 — MLOps for production AI systems · Module 4 — Active learning and feedback loops
Companion demonstrations	Demo 2 — Predictive Maintenance and Anomaly Triage · Demo 3 — Computer Vision for Industrial Quality Inspection
Format	Teams of four. Task 2 builds on Task 1's diagnosis; Tasks 3 and 4 can be done with any model, including one from an earlier day.
Toolchain	Azure Linux VM · GitHub Copilot (agent mode) · Python 3.11 · PyTorch CPU · scikit-learn · Streamlit. No GPU required today.
Every task ends with	A number that decides something — a label error rate, a cost-optimal threshold, a measured rollback time, a reviews-per-shift figure.

The claim today is built on. On industrial vision and machine-learning problems, the largest available gains are almost never in the model. They are in the labels, the capture conditions, and the loop that feeds production experience back into training. Day 1 gave you the diagnostic method; today applies it to the hardest industrial problem type and builds the machinery that keeps it running after everyone has moved on.

Task	Build	Where	What it makes undeniable
1	Why Does the Model Fail? Real inspection images	Lab 1	Between 10% and 30% of your "model failures" are labelling failures. You cannot know which without looking.
2	Error Analysis and Three Hypotheses Report plus one proven	Lab 2	The cost-optimal threshold needs no new data, no labels and no training run — and is often the largest single gain.
3	The MLOps Pipeline, Built Gate, registry, rollback	Lab 3	A gate that has never rejected anything is not a gate. Rollback time is a number or it is a hope.
4	The Feedback Loop, With Arithmetic Active learning, measured	Lab 4	Labelling the cases the model finds hard beats labelling at random, by a wide margin — but only if confidence is calibrated.

The four tasks together produce every one of the seven Day 4 assignment deliverables — see the closing page. Readouts are given at the start of the session following each lab.

1 · The virtual machine — and why there is no GPU today

Every experiment in this pack is designed to run on CPU in minutes, because the point of Day 4 is that the model is rarely the constraint. You will extract features once with a small pretrained network and then iterate on classifiers, thresholds, labels and sampling strategies in seconds. That is not a compromise for the classroom — it is the correct engineering order, and it is how you should approach the first fortnight of any real inspection project.

Size	<code>Standard_D4s_v5</code> — 4 vCPU, 16 GB RAM. Feature extraction over 8,000 images takes a few minutes; everything after that is seconds.
Image	Ubuntu Server 24.04 LTS · 64 GB Premium SSD
Networking	SSH (22) and Streamlit (8501) inbound, source restricted to the training-room public IP.

2 · The toolchain

```
source ~/.venv/bin/activate
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
pip install scikit-learn pillow opencv-python-headless pandas numpy scipy \
    streamlit plotly matplotlib mlflow jsonschema python-dotenv
mkdir -p ~/d4t1 ~/d4t2 ~/d4t3 ~/d4t4
```

Feature extraction	<code>torchvision</code> <code>mobilenet_v3_small</code> with pretrained weights, penultimate layer. About 5 ms per image on CPU. Extract once, cache to disk, reuse all day.
Classifier	<code>sklearn.linear_model.LogisticRegression</code> on those features. Trains in under a second, which is what makes forty experiments possible in a forty-five-minute lab.
Anomaly option	Mahalanobis distance on the features of good parts only — the "needs no defect examples" approach, in about twenty lines.
Experiment tracking	<code>mlflow</code> running locally with a file backend. No server, no account. <code>mlflow ui --host 0.0.0.0 --port 8501</code> .

3 · The dataset, and the licence check that comes first

Do this before you download anything, because it is the habit Module 1 asks you to build. Several widely-used vision frameworks and pretrained weights carry copyleft licences whose terms can extend to network-delivered services — which may require a commercial licence for an internal enterprise inspection service. Others restrict pretrained weights to non-commercial research use only.

Establish the licence position for every model and every dataset before it enters a pipeline. Today that means two checks, and they take four minutes:

1. Open the dataset page and record the stated licence, verbatim, with the date you checked it. Dataset licences change; a screenshot in your repository is worth having.
2. Record the licence of the pretrained weights you use. `torchvision` weights are BSD-3-Clause — permissive, commercially usable. Note it, so you can say so when asked.

Corpus	What it is
Surface crack images	40,000 images at 227×227 , split evenly between cracked and uncracked concrete surfaces. Özgenel & Gönenç Sorguç, Middle East Technical University, published via Mendeley Data (doi : 10 . 17632/5y9wdsq2zt . 2). Real photographs with genuine lighting and texture variation — which is what makes the failure analysis honest. Verify the current licence on the dataset page as described above. If it is not compatible with your use, the facilitator has a licence-clean synthetic generator that produces the same exercise.
Your corruptions	You will inject two things at known rates in Step 1 of Task 1: label noise, and capture-condition variation. Both are essential, and both are honest — because knowing the true rate is what lets you check whether your own audit found it.

4 · Working with Copilot — the discipline that separates the top teams

Three rules for Copilot use in these labs.

1. **Do not let it augment for you.** Copilot will offer a default augmentation pipeline with horizontal flips and 90-degree rotations because that is what every tutorial has. Every augmentation is a claim about your process, and most of those defaults are false claims about a fixed camera.
2. **Ask for the failures, not the score.** After it trains anything, prompt: *"Save the 50 highest-confidence false negatives and the 50 highest-confidence false positives as an image grid I can look at."* Looking at them is the lab.
3. **Make it separate the four version numbers.** In Task 3, ask explicitly for dataset version, feature-code version, model version and label-guideline version to be recorded together on every run. It will otherwise record the model alone.

5 · How your work is scored — identical for all four tasks

Criterion	Points	What full marks looks like
You looked at the failures	20	Individual images reviewed and categorised, not just metrics computed.
The constraint is named and evidenced	20	Labels, capture, coverage or model — one of them, with the evidence that rules out the other three.
Hypotheses are testable in advance	20	Expected effect size, statistical test and pass criterion written before the experiment ran.
The arithmetic works	20	Review volumes against real capacity; rollback time measured; cost curves from stated costs.
Something was rejected	10	A gate that blocked a model, a hypothesis that failed, an augmentation ruled out — anything that shows the process has teeth.
The verdict is actionable	10	An engineering manager could read one page and know what to fund.

Automatic deduction: minus 10 for any recommendation to collect more data without evidence ruling out labels and capture first; minus 10 for any accuracy figure reported without per-class breakdown. These are the two habits Day 4 exists to break.

Task 1 — Why Does the Model Fail?

Lab 1

MODULE 1

FAILURE TAXONOMY

PER-CLASS RECALL

THE LABEL AUDIT

CAPTURE CONDITIONS

REAL IMAGES

The scenario

You have a trained inspection model and a headline score that is not good enough. Everyone in the room already has an opinion about why. This lab exists to make you diagnose before you propose — and the finding that changes the conversation is almost never the one people expect.

Step 1 · Build the dataset you can actually audit

Take 8,000 images, balanced. Then inject two things **at rates you record and do not tell the rest of the room**:

Inject	At what rate	Why
Label noise — flip the label on a random subset	Pick something between 8% and 15%. Write it down and seal it.	Step 5 is a label audit. You need a true rate to check your own audit against, and real datasets have one you cannot measure.
Capture-condition groups — tag every image into one of four synthetic "shifts"	Apply a distinct transform per group: normal · low light (0.55× brightness) · glare (a bright spot) · slight blur	Step 6 groups failures by capture condition. Real production has these; here you know exactly which images belong to which.

Split 70/30, stratified, and keep the group and true-label columns out of training. Extract MobileNet features once and cache them; everything after this point runs in seconds.

Step 2 · The baseline, and the honest headline

Logistic regression on the cached features. Report **overall accuracy**, then immediately report **per-class recall** beside it. Put both on the dashboard and never show the first without the second again.

Step 3 · Look at fifty false negatives

Not fifty randomly sampled predictions — the fifty **highest-confidence** false negatives, rendered as an image grid you actually open. For each, tag it: small · low contrast · occluded · a specific class · nothing obviously wrong.

This step resists automation and teams resist doing it. Insist. An afternoon spent looking at two hundred failures reliably beats a month of hyperparameter tuning, and everyone who has done it once never skips it again. Copilot can build the grid; it cannot do the looking.

Step 4 · Look at fifty false positives

Same discipline. The question here is different: **is there a repeated benign feature the model has learned to treat as a defect?** A shadow at a fixture edge, a surface texture, a mark from the previous operation. If forty of your fifty false positives share one benign feature, you have found something a targeted fix will remove entirely.

Step 5 · The label audit — the step that changes minds

Take 30 of the failures you reviewed. For each, decide independently — **two team members, separately, without seeing the model's answer** — whether the recorded label is actually correct.

Record	Then
How many of the 30 were labelled wrongly	Compare against the true injection rate you sealed in Step 1. Did your audit find it?
How often your two reviewers disagreed	This is your ceiling. Compute Cohen's kappa on the 30.
The share of "model failures" that were label failures	Recompute your headline metric with the audited labels corrected. The gap is real and it was invisible.

What this typically shows, and why it changes the conversation with management. Somewhere between 10% and 30% of what a team reports as model failures are labelling failures. Every one of those is a case where more data makes things worse, a bigger model makes no difference, and the honest fix is a written guideline with boundary examples and a re-label of the affected subset.

Your model's realistic ceiling is roughly where your inspectors agree with each other. If two of you agreed 78% of the time on these 30 images, a 95% target is not ambitious — it is asking the model to be more consistent than the ground truth it learns from.

Step 6 · Group the failures by capture condition

Compute recall separately for each of your four condition groups. You will find one group carries a disproportionate share of the failures, and because you injected the groups you know exactly what physical change it corresponds to.

The point of doing it with known groups. In production these four axes — lighting, camera and optics, part presentation, product mix — are all changes to the *physical process* that nobody records as a change to the AI system. Each shows up as unexplained model degradation weeks later. The cheap recommendation, which costs nothing and is almost never in place: keep a change log linking physical process changes to model monitoring.

Step 7 · Name the constraint, in one sentence

The deliverable is a **ranked failure taxonomy with counts**, and one clear statement of which of four things is actually limiting you. Use the table below to rule out the other three rather than to pick a favourite.

If the situation is...	More data	Better labels	Better capture
Labels are inconsistent	no effect — adds inconsistency	large effect	no effect
Defect smaller than the model can resolve	no effect	no effect	large effect
A defect class has 12 examples	large — targeted collection	moderate	no effect
Lighting varies uncontrolled between shifts	small	no effect	large effect
Model overfits a small clean set	moderate	small	small
Two inspectors disagree on borderline parts	no effect	large effect	no effect

Four of these six rows are not solved by more data. Before anyone funds a collection programme, establish which row you are in.

Acceptance criteria

- ☐ Licence position recorded for both the dataset and the pretrained weights, with the date checked.
- ☐ Overall accuracy never shown without per-class recall beside it.
- ☐ 50 false negatives and 50 false positives individually reviewed and tagged — the grids available to show.
- ☐ Label audit on 30 failures by two independent reviewers, with kappa and the corrected metric.
- ☐ Recall broken down by capture-condition group.
- ☐ Ranked taxonomy with counts, and one sentence naming the constraint with the evidence that rules out the others.

Defend your result

"Our headline was ____%. Per class it was ____ and _____. Of 30 audited failures, ____ were labelling errors — we injected ____%, so our audit found ____ of it. Our two reviewers agreed at kappa _____. Recall on the ____ condition group was ____ points below the rest. The constraint is ____, and it is not more data because _____."

Task 2 — Error Analysis Report and Three Hypotheses

Lab 2

MODULE 2

COST-OPTIMAL THRESHOLD

PER-CLASS OPERATING POINTS

MCNEMAR

REVIEW CAPACITY

The scenario

Produce the report you would give your engineering manager, and the three hypotheses you would ask them to fund. Then — and this is what makes it a build rather than a document — **run one of the three and report whether it held.**

Step 1 · The report, seven sections

Section	Content
Failure taxonomy	Categories with counts, weighted by business cost, ranked
Per-class breakdown	Recall by class, with the two classes carrying most errors identified
Label audit	Of the failures reviewed, how many were labelling errors rather than model errors
Capture condition analysis	Failures grouped by lighting, camera, shift and product variant
Three hypotheses	Measurable, with expected effect size, statistical test and pass criterion set in advance
Threshold analysis	Cost of each error type and the cost-optimal operating point
Recommendation	Data, labels, capture or model — which one, and why the evidence supports it

Lead with the label audit. It is the line that usually changes the conversation, because it converts an argument about model quality into a finding about process quality — and it comes with a number.

Step 2 · Threshold and cost — the gain that needs nothing

No new data. No labels. No training run. On many industrial systems this is the largest single available gain, and it is the one teams check last.

1. **Establish both error costs.** A scrapped good part and an escaped defect. Use these placeholders and say they are placeholders: **€ 45** for a false reject, **€ 1,900** for an escape. Finance owns the real numbers.
2. **Sweep the threshold** on held-out data and plot total expected cost against operating point.
3. **Check per class.** The cost-optimal threshold frequently differs by defect class.
4. **Confirm on a later window.** A threshold tuned on old data may not hold — split your test set by index and check.

The idea most teams have not considered. Per-class thresholds. A safety-critical class set to favour recall heavily, a cosmetic class set to favour precision — one model, two operating points, no retraining. Implement it and report the cost difference against a single global threshold. It usually pays for the afternoon.

Step 3 · Write three hypotheses that could fail

Each one needs a measurable claim, a test chosen in advance, and a pass criterion. Use the pattern below; the fourth row is the one you will run today.

Observed failure	Measurable hypothesis	Test
Recall poor on small defects	Tiling at 2× overlap raises small-defect recall from 0.52 to ≥ 0.70	McNemar on the small-defect subset
Two classes confused	Re-labelling under a revised guideline raises per-class recall by ≥ 0.06 for both	Paired comparison on a fixed held-out set
Failures cluster on one condition group	500 additional images from that condition bring its recall within 0.03 of the rest	Two-proportion test between slices
False rejects too costly	Threshold at the cost-optimal point reduces total error cost by $\geq 30\%$	Cost curve on held-out data

Step 4 · Run one hypothesis properly

Pick the re-labelling hypothesis or the threshold hypothesis — both are achievable inside this lab. Run it as a genuine experiment, with the Day 1 apparatus:

- **Pre-register** the primary metric and pass criterion in a committed file before you run anything.
- **Compare with McNemar** — both models scored on the same images, each right or wrong per image. That is exactly McNemar's case, and an unpaired test here is the most common statistical error in vision work.
- **Report per slice.** An improvement that helps common classes and harms a rare critical one is not an improvement.
- **Report the effect size and its interval**, not only the p-value.

Expect at least one hypothesis to fail, and say so. A report where every proposed change worked is a report nobody experienced believes. The re-labelling hypothesis in particular often produces a smaller gain than teams expect, because only part of the label noise sits near the decision boundary — and that finding is worth knowing before you fund a full re-labelling programme.

Step 5 · The human review design, and its arithmetic

Any threshold that defers uncertain cases to a person creates a workload. Do the arithmetic before choosing the threshold, not after.

Quantity	Work it out
Line rate	2 parts per second
Shift length	8 hours
Parts per shift	compute it
Deferral rate at your chosen threshold	measure it
Reviews per shift	compute it — and compare against how many a person can actually do

The constraint that is far tighter than for documents. On a line running at two parts per second, a 12% deferral rate is roughly 8,600 reviews per shift. Review must fit inside the cycle time or the part must be diverted to an offline station. **A review design that exceeds capacity fails on day one** — and the threshold that produces it is not available to you, whatever the cost curve says.

Acceptance criteria

- ☐ Seven-section report complete, led by the label audit finding.
- ☐ Cost curve plotted from stated costs, with the cost-optimal point marked and the saving quantified.
- ☐ Per-class thresholds implemented and compared against a single global threshold.
- ☐ Three hypotheses with test and pass criterion pre-registered in a committed file.
- ☐ One hypothesis executed with McNemar, effect size and per-slice reporting — including if it failed.
- ☐ Review volume arithmetic completed against a stated reviewer capacity.

Defend your result

"The cost-optimal threshold saves € ____ per shift against 0.5, with no retraining. Per-class thresholds add a further € _____. We pre-registered _____ and it _____. At that operating point we defer _____% — which is _____ reviews per shift against a capacity of _____, so we _____."

Task 3 — The MLOps Pipeline, Built

Lab 3

MODULE 3

FOUR VERSIONS BOUND

EVALUATION GATE

REGISTRY

SHADOW DEPLOYMENT

ROLLBACK, TIMED

The scenario

MLOps is not a toolchain. It is five questions made answerable — and if the honest answer to any of them is no, that is your priority. Do not start with the tooling; start with the unanswered question. Buying a platform to solve a discipline problem is how organisations end up with an expensive platform and the same problem.

	The question	What you will build to answer it
1	Can you reproduce a result?	Four versions bound to every run
2	Can you roll back?	A registry and a rollback you will time with a stopwatch
3	Can you tell it degraded?	Monitoring signals, three of which need no labels
4	Can you ship a change safely?	An evaluation gate that will actually reject something today
5	Can you retrain reliably?	Triggers set in advance, and a validation gate before promotion

Step 1 · Version four things, and bind them together

What	Why	Consequence of omitting it
Dataset version	The exact data used for a run	Results cannot be reproduced or compared
Feature version	The code that computed the features	Training-serving skew; silent invalidation
Model version	Weights, architecture, hyperparameters	Cannot roll back; cannot say what is serving
Label guideline version	The definition the labels were created under	Labels from different eras silently incomparable

Why the binding matters more than the four numbers. Rolling back a model without rolling back the feature code gives you a model running on inputs it was never trained for — which frequently performs *worse* than either version alone. Teams discover this during an incident, which is the worst possible time. Every model version must record the dataset, feature and guideline versions that produced it, in one place, automatically.

Implement it as a content hash per artefact plus a `manifest.json` written by the training run. Register the model in MLflow with all four recorded as tags — the tags are the binding, and a run without them should fail.

Step 2 · Experiment tracking, with the field everyone skips

Record per run: dataset and feature versions, code commit, hyperparameters, all metrics **including per-class and per-slice**, duration and cost, who ran it — and **the hypothesis it was testing**.

The line that gets skipped and matters most. A tracked run without a stated hypothesis is a record that something was tried. A tracked run with one is a record of what was *learned* — including the negative results, which are precisely the ones teams repeat next year because nobody wrote down that they did not work.

Step 3 · The evaluation gate, and it must reject something today

Seven stages: commit · validate · train · evaluate · **gate** · register · deploy. The gate is what makes it MLOps rather than continuous integration, and its criteria are set **before** the run, exactly as on Day 1.

A gate that has teeth — write yours in this shape

PASS requires ALL of: primary metric > current production model (not "close to") no slice regresses by more than 0.02 (per class, per condition group) every regression test case passes (the failures you fixed before) calibration error < 0.05 (Task 4 depends on this) inference latency < the stated cycle-time budget FAIL → the model is not deployed, and it is not argued into production.

Prove your gate works by making it fail. Train a deliberately worse model — fewer training images, or the label noise left uncorrected — and push it through the pipeline. If it deploys, you have a logging statement, not a gate. **A gate that has never rejected anything is not a gate**, and this is the fastest way to find out which one you built.

This is also what stops quality drifting downward one small exception at a time, which is how it actually happens.

Step 4 · Registry, and the rollback you will time

The registry records version and lineage, metrics per class and per slice, approval status and approver, which environment it serves, and known limitations. It is the prerequisite for rollback.

The test of a good design, and it takes ninety seconds. How long from noticing a problem to the previous model serving again? **Time it with a stopwatch, from a cold start, with someone who did not build it doing the work.** Record the number. If nobody knows the number, that is the first gap to close — and if the number is "we would have to check with the person who set it up", that is the same answer.

Step 5 · Deployment strategy — and why shadow is unusually valuable here

Strategy	How it works	Detects
Shadow	New model runs live; its output is not used	Real behaviour, latency, cost and disagreement rate — at zero user risk
Canary	Small traffic share to the new model	Real outcomes on a limited population
Blue-green	Two environments, traffic switched	Nothing new — it is a fast rollback mechanism
Staged rollout	Progressive increase with monitoring	Problems emerging only at scale or on specific segments
Champion–challenger	Challenger runs continuously alongside	Sustained comparison rather than a one-off test

Implement **shadow**: run both models on the same held-out stream, log every disagreement, and report the disagreement rate and the cases where they differ. For industrial vision this is unusually valuable, because you can compare the new model's decisions against the current one *and* against inspector judgement on the same physical parts, before anything changes on the line.

Step 6 · Monitoring — and the three signals that need no labels

Signal	Indicates	Without labels?
Input distribution shift	Capture conditions or product mix changed	Yes — the earliest signal
Confidence distribution shift	The model is less certain	Yes
Deferral rate rising	More cases in the uncertain band	Yes
Reviewer override rate rising	Humans disagreeing more often	Yes, if captured
Accuracy on a fresh sample	Actual performance	No — requires labelling
Business outcome change	Escapes, scrap rate, warranty claims	Lagging, but the one that matters

Demonstrate the first one: feed your **low-light condition group** through as if it were a week of production, and show the input-distribution monitor firing *before* any labelled accuracy figure could have existed.

Step 7 · Retraining triggers, set in advance

Accuracy below the error budget on the fresh sample · a defined volume of new labelled data · a logged physical process change · or a scheduled cadence as a floor.

Triggering on a single alarming week is how you retrain on noise. Every trigger needs a threshold and a duration, not just a threshold. And a physical process change — a replaced camera, a moved lighting rig, a new supplier — should trigger review even when every metric still looks fine, because it is a distribution shift nobody logged.

Acceptance criteria

- ☐ All four versions recorded and bound on every run, automatically, with a run missing them failing.
- ☐ Every tracked run carries the hypothesis it was testing.
- ☐ The gate rejected a deliberately worse model, in front of the facilitator.
- ☐ Rollback timed with a stopwatch by someone who did not build it, and the number recorded.
- ☐ Shadow deployment running, with a disagreement rate reported.
- ☐ Input-distribution monitor demonstrated firing on the shifted condition group.
- ☐ Retraining triggers written with both a threshold and a duration.

Defend your result

"Our gate rejected a model that scored ____ because _____. Rollback took ____ seconds, done by ____ who had not seen the pipeline. Shadow shows the two models disagreeing on ____% of parts, concentrated in _____. Our earliest degradation signal is _____, and it fires _____ before any labelled accuracy figure could."

The scenario

From a low-confidence prediction to a retrained, validated, deployed model. You will build the loop, and — more importantly — you will show with a measurement that it is worth building, and with arithmetic that it can actually run.

Step 1 · Check calibration before anything else

Uncertainty sampling selects the cases the model is least sure about. That only works if the model's confidence means something.

The dependency that is easily missed and undermines everything. If the model is over-confident — which most neural networks are — it will select the wrong examples, and the whole loop underperforms for a reason nobody diagnoses. Plot the reliability curve and compute the calibration error **first**. If it is poor, apply Platt scaling or isotonic regression on a held-out slice and re-measure. Then proceed.

Report calibration error before and after. It is a two-line fix and it is the difference between an active-learning loop that pays and one that quietly does not.

Step 2 · The experiment — is active learning actually worth it?

This is the measurement that justifies the whole task, and it runs in about a minute on cached features.

1. Start from a small seed training set — say 200 labelled images.
2. **Arm A — random.** Add 100 randomly chosen images, retrain, score. Repeat to 1,200.
3. **Arm B — uncertainty.** Add the 100 the current model is least certain about, retrain, score. Repeat to 1,200.
4. Plot both learning curves: accuracy against number of labels bought.

What you are demonstrating, in a form a budget holder understands. Read off the horizontal distance between the curves at your target accuracy. That distance is **labels you did not have to pay for**. Random labelling spends most of its budget on easy cases the model already handles; every uncertainty-selected label lands where the decision boundary is unclear.

The apprentice. You would not train an apprentice by having them repeat tasks they already do correctly. You would put them on the jobs they find difficult, with someone watching. Active learning is that principle applied to a training set.

Add a third arm if you have time: **error-based sampling**, using the cases where your Task 1 label audit overturned the model. Those are the highest-information examples available, and they come free from a review loop you are already running.

The link back to Day 1, and the limit it implies. Active learning targets *epistemic* uncertainty — the kind more data reduces. It will not help with *aleatoric* uncertainty. If the model is uncertain because the task is genuinely ambiguous, labelling those cases adds contradictory examples rather than information. Your Task 1 kappa tells you how much of your uncertainty is which, and therefore whether the loop will pay.

Step 3 · The routing rule and the volume arithmetic

Define which confidence range goes to review, plus any always-review classes. Then do the arithmetic, on the diagram, in numbers.

Quantity	Your number
Parts inspected per shift	
Deferral rate at your routing threshold	
Reviews per shift	
Seconds available per review	
Reviewer capacity actually available	
Does it fit?	If not, the design does not work — change the threshold or divert the part

Step 4 · The capture schema — the step teams skip

Teams build the review interface and never store the decision in a form that can reach training, so the most valuable data the system generates is discarded daily. Define the schema explicitly and write to it.

Capture	Why
What the reviewer decided	The corrected label
What the model proposed, and its confidence	Lets you measure where and how the model is wrong, not just that it was
Where they disagreed	The information-carrying subset
Which reviewer, when, under which guideline version	Attribution — and labels from different guideline versions are not comparable
Whether it was reconciled, and by whom	Unreconciled disagreement is training noise

Make correcting easier than overriding. If correcting takes four clicks and overriding takes one, you will get overrides and no training data. **The interface determines the data quality** — this is an engineering decision disguised as a UX detail. Build the interface so the correction is the default path.

Step 5 · The validation gate, and the leakage rule

Teams retrain on newly collected data without an evaluation gate, so a batch of mislabelled reviews degrades the model with nobody noticing. Reuse the gate from Task 3 and add one rule that is easy to get wrong:

Each new labelled case goes to training or evaluation — never both. Newly labelled production cases flowing into both quietly inflates every future measurement, and it is Day 1's leakage lesson reappearing in operational form. Write the split at capture time, not at training time, and record it in the capture schema.

Also: keep a **protected evaluation set** that is never touched by the loop, score it on a fixed schedule, and refresh it from production only deliberately and with an expert verifying every added case.

Step 6 · Close the loop visibly

The point that decides whether the loop still runs in six months. Tell reviewers when their corrections improved the model — which cases, how much, on what. Correction rates decay within weeks if reviewers cannot see that it mattered. This is a human factor and it is decisive, more so than any technical choice in the loop. Put it in your design as a named element with an owner, not as an aspiration.

Acceptance criteria

- ☐ Calibration measured before relying on confidence, and corrected if poor — with both numbers reported.
- ☐ Random versus uncertainty learning curves plotted, with the label saving read off at a stated target.
- ☐ Loop diagram carrying the volume arithmetic on it.
- ☐ Capture schema defined and written to, including guideline version and reviewer attribution.
- ☐ Train-or-evaluate split assigned at capture time, demonstrably not both.
- ☐ Reviewer feedback named as an element with an owner.

Defend your result

"Calibration error was ___ and is now ___. To reach ___% accuracy, random sampling needed ___ labels and uncertainty sampling needed ___ — a saving of ___. At our routing threshold that is ___ reviews per shift against a capacity of ___. Our loop stores ___ per review, and new cases go to training or evaluation but never both because ___."

The Day 4 assignment, and where each piece comes from

Reference

Assignment deliverable		Where you built it
1	CV or ML error analysis	Task 1 — ranked failure taxonomy with counts, per-class recall, and the label audit result
2	Improvement hypotheses and validation	Task 2, Steps 3 and 4 — three measurable hypotheses with test, sample size and pass criterion set in advance
3	Active learning strategy	Task 4, Steps 1 and 2 — the sampling strategy chosen and justified, with the calibration check completed
4	Feedback loop design	Task 4, Steps 3–6 — routing rule, volume arithmetic, capture schema and reviewer feedback
5	MLOps pipeline diagram	Task 3 — versioning, training, evaluation gate, registry, deployment and rollback
6	Monitoring and retraining plan	Task 3, Steps 6 and 7 — signals, thresholds, alert routing and the specific triggers
7	Expected accuracy improvement roadmap	Tasks 1, 2 and 4 combined — what you expect each intervention to deliver, with the evidence behind the estimate

Scored on diagnostic rigour, hypothesis quality, loop design realism and operational completeness.

Augmentation — the test to apply before you use any of it

Augmentation teaches the model what to **ignore**. Every augmentation is a claim about your process.

Legitimate — variation that genuinely occurs	Illegitimate — a false claim about your process
Brightness and contrast within the real range	90-degree rotation with a fixed camera and fixture
Small rotations, if placement genuinely varies	Horizontal flip on an asymmetric part with fixed orientation
Noise and blur matching real sensor behaviour	Colour shifts on a monochrome setup
Scale, if working distance varies	Elastic distortion on a rigid metal component

The practical test: could this image plausibly have come out of your camera? Teaching a model to ignore orientation when a mark on the left means something different from a mark on the right actively destroys information it needs. Check the claim with someone who knows the line before applying it to a million images.

When the honest fix is optical, not algorithmic

Condition	Why the model fails	What actually fixes it
Small object	Downsampled out of existence before the detection head sees it	Resolution, tiling, or higher optical magnification
Partial occlusion	The distinguishing feature is hidden	Change the fixture or add a viewpoint
Low contrast	Defect and surface differ by few grey levels	Directional or raking lighting, polarisers
Specular reflection	Highlights saturate the sensor	Polarisers, diffuse lighting, multiple exposures
Motion blur	The part moves during exposure	Shorter exposure with more light, or a strobe
Depth of field	Part of the surface is out of focus	Smaller aperture with more light, or telecentric optics
Parallax on a gauge	Camera not perpendicular — reads consistently high or low	A mounting bracket. Systematic error; averaging does not help

The diagnostic that costs an hour and settles the question. Can a trained inspector reliably see the defect in this image? If not, the model cannot either, and no architecture or volume of data will change that. Print twenty failure images and ask an inspector. **Also ask: how many pixels does the smallest defect you must catch occupy?** If the answer is under about twenty, the honest fix is optical rather than algorithmic.

Data and references

Item	Reference
Surface crack image dataset Tasks 1, 2, 4	Özgenel, Ç.F. & Gönenç Sorguç, A. (2018), "Performance Comparison of Pretrained Convolutional Neural Networks on Crack Detection in Buildings", ISARC 2018. Published via Mendeley Data, doi:10.17632/5y9wdsq2zt.2 . 40,000 images at 227×227 , two classes. Verify the current licence on the dataset page and record it with the date checked — this is Step 0 of Task 1, not a formality.
Pretrained weights	<code>torchvision</code> <code>MobileNet_V3_Small_Weights.IMAGENET1K_V1</code> — BSD-3-Clause, permissive and commercially usable. Record it alongside the dataset licence.
Cost figures	€ 45 per false reject and € 1,900 per escape are illustrative placeholders. Finance owns the real numbers, and every threshold in your report moves when they change.

On data hygiene. No Atlas Copco images, inspection records or production data should be placed on a training virtual machine. The public dataset stands in for your own inspection images; the failure modes it exhibits — lighting variation, texture confusion, ambiguous boundaries — are the same ones yours will.

The five sentences worth taking out of the room

- 1 Diagnose before collecting — four of six common situations are not solved by more data.
- 2 Label consistency sets the ceiling. Your model cannot be more consistent than the ground truth it learns from.
- 3 Every augmentation is a claim about your process.
- 4 The evaluation gate is what makes MLOps real — and a gate that has never rejected anything is not a gate.
- 5 A feedback loop only compounds if capture and validation are both present. Most have neither.

Before you leave today

Name, as a team, your label agreement rate. Most teams do not know it. That is the honest finding to carry into the capstone tomorrow — and it is usually measurable in an afternoon.